REPORTE: OPTIMIZACIÓN MEDIANTE COLONIA DE ABEJAS ARTIFICIALES (ABC)

Materia: Algoritmos Metaheurísticos

Actividad: 4 - Minimización de la Función Eggholder

1. Marco Teórico

El algoritmo de **Colonia de Abejas Artificiales (ABC)** es una metaheurística inspirada en el comportamiento inteligente de las abejas al buscar néctar. El algoritmo divide la población en tres grupos:

1. **Abejas Obreras:** Explotan fuentes de alimento conocidas y realizan búsquedas locales para encontrar mejores flores en la vecindad.
2. **Abejas Observadoras:** Esperan en la colmena y eligen las mejores fuentes basándose en el "baile" de las obreras. Utilizan una selección por **ruleta**, lo que garantiza que las zonas con más néctar sean exploradas más intensamente.
3. **Abejas Exploradoras:** Cuando una fuente se agota (estancamiento), la abeja abandona ese lugar y vuela al azar para encontrar una fuente totalmente nueva, evitando así quedar atrapada en mínimos locales.

2. Desarrollo (Fragmentos Clave del Código)

A. Función Objetivo: Eggholder

La función Eggholder es un reto para la optimización debido a su gran cantidad de mínimos locales.

```
double get_eggholder(double x, double y) {
        double t1 = -(y + 47.0) * sin(sqrt(abs(x / 2.0 + (y + 47.0))));
        double t2 = -x * sin(sqrt(abs(x - (y + 47.0))));
        return t1 + t2;
    }
```

B. Conversión a Fitness (Maximización de Néctar)

Para que el algoritmo funcione, transformamos el **costo** (que queremos minimizar) en **fitness** (néctar, que las abejas quieren maximizar).

```
    // El fitness (néctar) debe ser positivo. Esta formula asegura que
    // a menor costo (valor mas negativo), mayor sea el fitness para la ruleta.
    double calc_fitness(double f) {
        return (f >= 0) ? (1.0 / (1.0 + f)) : (1.0 + abs(f));
    }
```

C. Movimiento y Búsqueda Local

Esta es la fórmula de perturbación que permite a la abeja saltar a una posición vecina.

```
// Formula Zi = Xi + phi * (Xi - Yk): Se mueve un poco hacia o lejos del vecino
            nueva.coords[j] = fuentes[i].coords[j] + phi * (fuentes[i].coords[j] - fuentes[k].coords[j]);
```

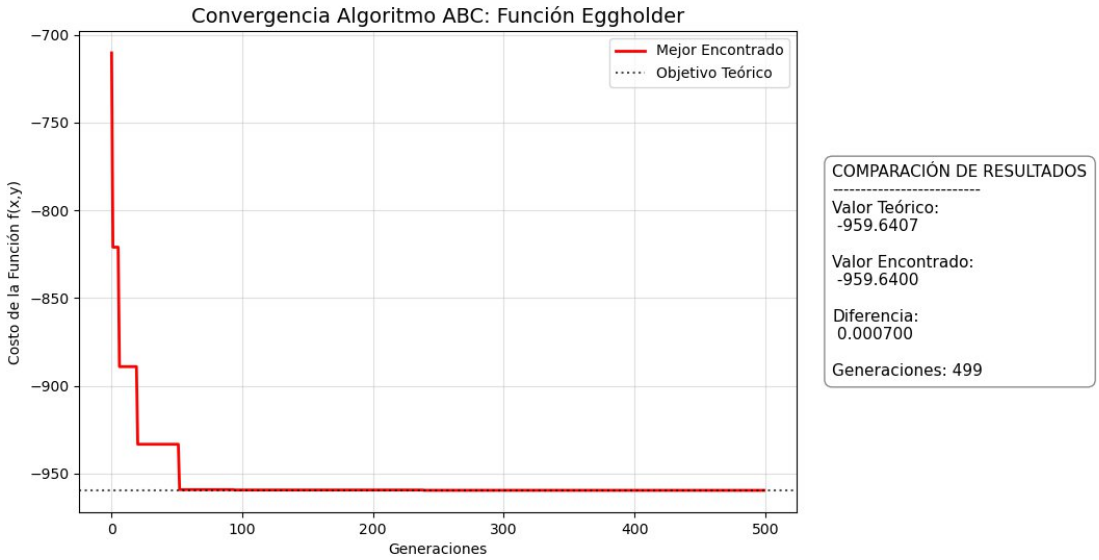
D. Fase de Exploradoras (Evitar Estancamiento)

Si tras `L` intentos una fuente no mejora, se reinicia aleatoriamente.

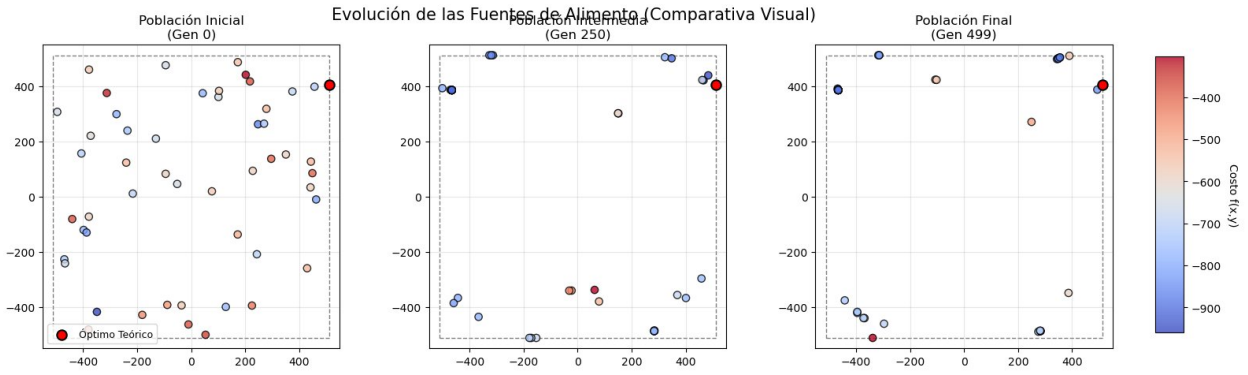
```
            if (fuentes[i].intentos > limite_L) {
                // La abeja olvida la fuente estancada y busca una nueva al azar (Exploracion Global)
                fuentes[i].coords = {dist(motor), dist(motor)};
                fuentes[i].res = get_eggholder(fuentes[i].coords[0], fuentes[i].coords[1]);
                fuentes[i].fitness = calc_fitness(fuentes[i].res);
                fuentes[i].intentos = 0;
            }
```

3. Resultados Gráficos

1. **Gráfica de Fitness (Convergencia):** Muestra cómo el valor desciende rápidamente desde valores cercanos a -600 hasta alcanzar el óptimo de **-959.64**.



2. **Representación de Poblaciones:**
   - **Inicial:** Abejas dispersas por todo el plano de -512 a 512.
   - **Intermedia:** Las abejas comienzan a agruparse en los "valles" más profundos.
   - **Final:** La mayoría de las abejas se concentran cerca del óptimo (512, 404.23), aunque algunas exploradoras se verán dispersas buscando otras opciones.



4. Conclusiones

En esta actividad se logró implementar con éxito el algoritmo ABC para minimizar una de las funciones más complejas de la optimización: **Eggholder**.

Se observó que el parámetro de **límite de estancamiento (L)** es crucial; sin él, las abejas se quedan atrapadas en mínimos locales rápidamente. La transformación del costo a un valor de fitness positivo permitió que la selección por ruleta de las abejas observadoras funcionara de forma eficiente, concentrando el esfuerzo de búsqueda en las regiones más prometedoras del espacio de soluciones.

El algoritmo convergió satisfactoriamente al valor teórico de **-959.64**, demostrando la potencia de las metaheurísticas de enjambre frente a problemas con paisajes rugosos.

5. Referencias

- Karaboga, D., & Basturk, B. (2007). *A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm*.
- Materia: Algoritmos Metaheurísticos - Apuntes sobre Colonia de Abejas Artificiales.
- Documentación técnica de la función Eggholder (Virtual Library of Simulation Experiments).

https://en.wikipedia.org/wiki/Artificial_bee_colony_algorithm

https://www.sfu.ca/~ssurjano/egg.html

6. Código completo

C++ :

```cpp
#include <iostream>

#include <vector>

#include <cmath>

#include <random>

#include <algorithm>

#include <fstream>

#include <string>
```

```cpp
#include <iomanip>


using namespace std;


const double PI = 3.141592653589793;


// Estructura para representar cada fuente de alimento (las flores con néctar)
struct Fuente {
    vector<double> coords; // Xi: Posicion en el mapa (x, y)
    double res;        // Valor real de la funcion Eggholder, es el costo que queremos minimizar
    double fitness;          // Calidad del nectar (transformado para que mas sea mejor)
    int intentos;          // To: Contador para saber si la fuente se agoto (estancamiento)
    double p;              // Probabilidad de que una observadora elija esta fuente
};


class BeeColony {
private:
    int dimensiones = 2; // Problema 2D (x, y)
    double lim_inf, lim_sup;
    int pop, n_obreras, n_observadoras;
    int limite_L;    // L: Numero maximo de fallos permitidos antes de abandonar una fuente
    int iteraciones; // Generaciones totales del algoritmo
```

```
    vector<Fuente> fuentes; // Conjunto de soluciones actuales (fuentes de néctar)
    mt19937 motor;          // Generador de numeros aleatorios Mersenne Twister
    Fuente best;            // La mejor fuente encontrada en toda la ejecucion


public:
    BeeColony(int n_abejas, int L, double min, double max, int gens) {
        pop = n_abejas;
        limite_L = L;
        lim_inf = min;
        lim_sup = max;
        iteraciones = gens;

        // La colonia se divide: la mitad explota y la otra mitad observa el baile
        n_obreras = pop / 2;
        n_observadoras = pop / 2;

        random_device rd;
        motor = mt19937(rd());
        best.res = 1e9; // Valor inicial muy alto para buscar el minimo
    }

    // Funcion Eggholder: El paisaje donde las abejas buscan el punto mas bajo
    double get_eggholder(double x, double y) {
        double t1 = -(y + 47.0) * sin(sqrt(abs(x / 2.0 + (y + 47.0))));
```

```
        double t2 = -x * sin(sqrt(abs(x - (y + 47.0))));
        return t1 + t2;
    }

    // El fitness (néctar) debe ser positivo. Esta formula asegura que
    // a menor costo (valor mas negativo), mayor sea el fitness para la ruleta.
    double calc_fitness(double f) {
        return (f >= 0) ? (1.0 / (1.0 + f)) : (1.0 + abs(f));
    }

    // Inicializacion: Las abejas salen por primera vez a buscar comida al azar
    void init() {
        fuentes.clear();
        uniform_real_distribution<double> dist(lim_inf, lim_sup);

        for (int i = 0; i < n_obreras; i++) {
            Fuente f;
            f.coords = {dist(motor), dist(motor)}; // Ubicacion aleatoria [-512, 512]
            f.res = get_eggholder(f.coords[0], f.coords[1]);
            f.fitness = calc_fitness(f.res);
            f.intentos = 0; // Fuente nueva, tiene 0 fallos acumulados
            fuentes.push_back(f);

            if (f.res < best.res) best = f;
        }
```

```
    }


    // Fase de obreras: Busqueda local. Cada abeja intenta mejorar su
propia fuente.
    void faseObreras() {
        uniform_int_distribution<int> d_vecino(0, n_obreras - 1);
        uniform_int_distribution<int> d_dim(0, dimensiones - 1);
        uniform_real_distribution<double> d_phi(-1.0, 1.0);


        for (int i = 0; i < n_obreras; i++) {
            // Se elige una abeja vecina k al azar para comparar (k != i)
            int k = d_vecino(motor);
            while (k == i) k = d_vecino(motor);


            int j = d_dim(motor); // Se altera solo una dimension (x o y)
            Fuente nueva = fuentes[i];
            double phi = d_phi(motor); // El factor de mutacion (S)


            // Formula Zi = Xi + phi * (Xi - Yk): Se mueve un poco hacia o
lejos del vecino
            nueva.coords[j] = fuentes[i].coords[j] + phi *
(fuentes[i].coords[j] - fuentes[k].coords[j]);


            // Verificacion de bordes
            if (nueva.coords[j] < lim_inf) nueva.coords[j] = lim_inf;
            if (nueva.coords[j] > lim_sup) nueva.coords[j] = lim_sup;


            nueva.res = get_eggholder(nueva.coords[0], nueva.coords[1]);
```

```
            nueva.fitness = calc_fitness(nueva.res);


            // Seleccion codiciosa: solo nos quedamos con el cambio si es
mejor
            if (nueva.fitness > fuentes[i].fitness) {
                fuentes[i] = nueva;
                fuentes[i].intentos = 0; // Exito: reiniciamos contador de
estancamiento
            } else {
                fuentes[i].intentos++; // Fallo: la fuente se acerca a ser
abandonada
            }
        }
    }


    // Fase de observadoras: Eligen las mejores fuentes mediante una
ruleta de probabilidad
    void faseObservadoras() {
        double total_fitness = 0;
        for (int i = 0; i < n_obreras; i++) total_fitness +=
fuentes[i].fitness;
        // Calculo de probabilidad: a mas nectar, mas abejas atrae la
fuente
        for (int i = 0; i < n_obreras; i++) fuentes[i].p =
fuentes[i].fitness / total_fitness;


        uniform_real_distribution<double> d_rand(0, 1); // Probabilidad
para la ruleta
        uniform_int_distribution<int> d_vecino(0, n_obreras - 1);
        uniform_int_distribution<int> d_dim(0, dimensiones - 1);
```

```
        uniform_real_distribution<double> d_phi(-1.0, 1.0);


        int count = 0;
        // Las observadoras se asignan a las fuentes mas prometedoras
        while (count < n_observadoras) {
            for (int i = 0; i < n_obreras && count < n_observadoras; i++)
{
                // Metodo de ruleta
                if (d_rand(motor) < fuentes[i].p) {
                    count++;


                    int k = d_vecino(motor);
                    while (k == i) k = d_vecino(motor);


                    int j = d_dim(motor);
                    Fuente nueva = fuentes[i];
                    double phi = d_phi(motor); //S en la lección


                    // Busqueda local intensiva en las mejores zonas
                    nueva.coords[j] = fuentes[i].coords[j] + phi *
(fuentes[i].coords[j] - fuentes[k].coords[j]);


                    if (nueva.coords[j] < lim_inf) nueva.coords[j] =
lim_inf;
                    if (nueva.coords[j] > lim_sup) nueva.coords[j] =
lim_sup;


                    nueva.res = get_eggholder(nueva.coords[0],
nueva.coords[1]);
```

```
                    nueva.fitness = calc_fitness(nueva.res);


                    if (nueva.fitness > fuentes[i].fitness) {
                        fuentes[i] = nueva;
                        fuentes[i].intentos = 0;
                    } else {
                        fuentes[i].intentos++;
                    }
                }
            }
        }


    // Fase de exploradoras: Si una fuente no mejora tras L intentos, se abandona
    void faseExploradoras() {
        uniform_real_distribution<double> dist(lim_inf, lim_sup);
        for (int i = 0; i < n_obreras; i++) {
            if (fuentes[i].intentos > limite_L) {
                // La abeja olvida la fuente estancada y busca una nueva al azar (Exploracion Global)
                fuentes[i].coords = {dist(motor), dist(motor)};
                fuentes[i].res = get_eggholder(fuentes[i].coords[0], fuentes[i].coords[1]);
                fuentes[i].fitness = calc_fitness(fuentes[i].res);
                fuentes[i].intentos = 0;
            }
        }
```

```
    }

    // Actualiza el record del mejor valor encontrado hasta el momento
    void check_best() {
        for(int i = 0; i < n_obreras; i++) {
            if (fuentes[i].res < best.res) best = fuentes[i];
        }
    }

    void run() {
        init(); // Generacion 0

        // Exportacion de datos para graficar en Python
        ofstream f1("../results/csv/fit_Eggholder_ABC.csv");
        ofstream f2("../results/csv/pob_Eggholder_ABC.csv");

        f1 << "Gen,MejorCosto" << endl;
        f2 << "Gen,X,Y,Costo" << endl;

        for (int g = 0; g < iteraciones; g++) {
            // Guardamos muestras de la poblacion (inicial, media, final)
            if (g == 0 || g == iteraciones / 2 || g == iteraciones - 1) {
                for (int i = 0; i < n_obreras; i++) {
                    f2 << g << "," << fuentes[i].coords[0] << "," <<
fuentes[i].coords[1] << "," << fuentes[i].res << endl;
                }
            }
```

```
            faseObreras();      // Explotacion individual
            faseObservadoras(); // Explotacion colectiva/selectiva
            check_best();       // Guardar record
            faseExploradoras(); // Salir de minimos locales (Exploracion)

            f1 << g << "," << best.res << endl;
        }

        f1.close(); f2.close();

        cout << "Optimo Eggholder encontrado: " << fixed <<
setprecision(6) << best.res << endl;
        cout << "Coordenadas del hallazgo: (" << best.coords[0] << ", " <<
best.coords[1] << ")" << endl;
    }
};

int main() {
    // Preparacion de carpetas
    system("mkdir -p ../results/csv ../results/plots");

    cout << "--- Algoritmo de Colonia de Abejas Artificiales ---" << endl;

    // Configuracion: 100 abejas, Limite L=100, Rango [-512, 512], 500
iteraciones
    BeeColony colmena(100, 100, -512.0, 512.0, 500);
    colmena.run();
```

```
    // Ejecucion automatica del script de visualizacion
    cout << "Generando visualizaciones..." << endl;
    system("cd ../results/csv && python3 ../../src/graficar_abc.py");

    return 0;
}
```

Python (Gráficas) :

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import os

# Asegurar que el directorio de gráficas exista
os.makedirs("../plots", exist_ok=True)

print("Generando graficas actualizadas...")

# 1. VALORES PARA COMPARACIÓN
# El valor teórico es una constante conocida para la función Eggholder
valor_teorico = -959.6407

# 1. GRÁFICA DE CONVERGENCIA CON COMPARACIÓN

archivo_fit = "fit_Eggholder_ABC.csv"
```

```
if os.path.exists(archivo_fit):

    df_fit = pd.read_csv(archivo_fit)

    mejor_encontrado = df_fit['MejorCosto'].iloc[-1]

    gen_final = df_fit['Gen'].iloc[-1]



    plt.figure(figsize=(12, 6))

    plt.plot(df_fit['Gen'], df_fit['MejorCosto'], color='red',
linewidth=2, label='Mejor Encontrado')



    # Línea horizontal para representar el objetivo teórico

    plt.axhline(y=valor_teorico, color='black', linestyle=':', alpha=0.7,
label='Objetivo Teórico')



    plt.title('Convergencia Algoritmo ABC: Función Eggholder',
fontsize=14)

    plt.xlabel('Generaciones')

    plt.ylabel('Costo de la Función f(x,y)')

    plt.grid(True, alpha=0.4)

    plt.legend(loc='upper right')



    # Ajustar margen para el cuadro de comparación

    plt.subplots_adjust(right=0.7)



    # Cuadro de comparación detallado

    texto_comparativo = (

        f"COMPARACIÓN DE RESULTADOS\n"

        f"--------------------------\n"

        f"Valor Teórico:\n {valor_teorico:.4f}\n\n"
```

```
        f"Valor Encontrado:\n {mejor_encontrado:.4f}\n\n"
        f"Diferencia:\n {abs(valor_teorico - mejor_encontrado):.6f}\n\n"
        f"Generaciones: {int(gen_final)}"
    )

    plt.text(1.05, 0.5, texto_comparativo, transform=plt.gca().transAxes,
             fontsize=11, verticalalignment='center',
             bbox=dict(boxstyle="round,pad=0.5", facecolor="white",
edgecolor="gray", alpha=0.9))

    plt.savefig("../plots/Conv_Eggholder_ABC.png", dpi=100)
    plt.close()

# 2. GRÁFICA DE POBLACIÓN (REPARACIÓN DE BARRA DE COLOR)

archivo_pob = "pob_Eggholder_ABC.csv"

if os.path.exists(archivo_pob):
    df_pob = pd.read_csv(archivo_pob)
    gens = sorted(df_pob['Gen'].unique())

    # Aumentamos el ancho total para dar espacio real a la barra lateral
    fig, axes = plt.subplots(1, len(gens), figsize=(6 * len(gens), 5))
    if len(gens) == 1: axes = [axes]

    nombres_fases = ["Inicial", "Intermedia", "Final"]
```

```
    for i, gen in enumerate(gens):
        data = df_pob[df_pob['Gen'] == gen]

        # Graficar abejas
        im = axes[i].scatter(data['X'], data['Y'], c=data['Costo'],
cmap='coolwarm',
                             s=45, alpha=0.8, edgecolor='k', zorder=2)

        titulo_fase = nombres_fases[i] if i < len(nombres_fases) else
f"Gen {gen}"
        axes[i].set_title(f'Población {titulo_fase}\n(Gen {gen})')

        # Encuadre
        axes[i].set_xlim(-550, 550)
        axes[i].set_ylim(-550, 550)

        # Marco del espacio de búsqueda
        rect = patches.Rectangle((-512, -512), 1024, 1024, linewidth=1,
                                 edgecolor='gray', facecolor='none',
linestyle='--', zorder=1)
        axes[i].add_patch(rect)

        # Óptimo teórico (marcador ajustado)
        axes[i].scatter(512, 404.2319, marker='o', s=80, facecolor='red',
                        edgecolor='black', linewidth=1.5, label='Óptimo
Teórico', zorder=10)

        if i == 0:
```

```
            axes[i].legend(loc='lower left', fontsize=9)

        axes[i].grid(True, alpha=0.3)


    # CORRECCIÓN DE LA BARRA:

    # subplots_adjust libera espacio a la derecha y fig.colorbar usa ese espacio específico

    plt.subplots_adjust(right=0.88, wspace=0.3)

    cax = fig.add_axes([0.91, 0.15, 0.02, 0.7]) # [left, bottom, width, height]

    cbar = fig.colorbar(im, cax=cax)

    cbar.set_label('Costo f(x,y)', rotation=270, labelpad=15)


    plt.suptitle('Evolución de las Fuentes de Alimento (Comparativa Visual)', fontsize=15, y=0.98)


    plt.savefig("../plots/Pob_Eggholder_ABC.png", dpi=100, bbox_inches='tight')

    plt.close()


print("✓ Gráficas corregidas. Se añadió cuadro de comparación y se movió la barra de color.")
```